

# picoacq

## Data Acquisition User Manual

### Coherent multi-channel HF capture with the PicoScope 5444D for triloop polarimetry

May 23, 2026

Version 0.1.0

## Contents

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Overview</b>                                          | <b>2</b> |
| 1.1      | What picoacq does . . . . .                              | 2        |
| 1.2      | Audience . . . . .                                       | 2        |
| <b>2</b> | <b>Hardware setup</b>                                    | <b>3</b> |
| 2.1      | The PicoScope 5444D . . . . .                            | 3        |
| 2.2      | The crucial connection check . . . . .                   | 3        |
| 2.3      | Sanity-test with the GUI app . . . . .                   | 3        |
| <b>3</b> | <b>Software installation</b>                             | <b>3</b> |
| 3.1      | Layer 1: PicoSDK system library . . . . .                | 4        |
| 3.2      | Layer 2: picosdk Python wrapper . . . . .                | 4        |
| 3.3      | Layer 3: picoacq itself . . . . .                        | 4        |
| 3.4      | Verifying the install . . . . .                          | 4        |
| <b>4</b> | <b>Hardware capabilities and resolution selection</b>    | <b>5</b> |
| <b>5</b> | <b>Channel configuration</b>                             | <b>5</b> |
| 5.1      | Voltage range selection . . . . .                        | 5        |
| 5.2      | Auto-ranging . . . . .                                   | 6        |
| 5.3      | Coupling . . . . .                                       | 7        |
| 5.4      | Channel-name to PicoScope-port mapping . . . . .         | 7        |
| <b>6</b> | <b>Sample-rate selection</b>                             | <b>7</b> |
| 6.1      | Common sample rates and their timebase indices . . . . . | 7        |
| 6.2      | Memory budget . . . . .                                  | 7        |
| <b>7</b> | <b>Command-line interface</b>                            | <b>8</b> |
| 7.1      | One-shot capture: picoacq capture . . . . .              | 8        |
| 7.2      | Continuous monitoring: picoacq monitor . . . . .         | 8        |
| 7.3      | Simulator mode: picoacq capture --simulate . . . . .     | 9        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>8</b>  | <b>HDF5 file format</b>                                  | <b>9</b>  |
| 8.1       | Reading a capture file in Python . . . . .               | 10        |
| <b>9</b>  | <b>Worked examples</b>                                   | <b>10</b> |
| 9.1       | Example 1: smoke-test on the bench . . . . .             | 10        |
| 9.2       | Example 2: 60-second WWV capture . . . . .               | 11        |
| 9.3       | Example 3: continuous overnight monitor at APO . . . . . | 11        |
| <b>10</b> | <b>Troubleshooting</b>                                   | <b>11</b> |
| 10.1      | Common errors and what they mean . . . . .               | 11        |
| 10.2      | Verifying timebase availability manually . . . . .       | 12        |
| <b>11</b> | <b>Hardware capture process, step by step</b>            | <b>13</b> |
| <b>12</b> | <b>Limitations of v0.1</b>                               | <b>13</b> |
| <b>13</b> | <b>Quick reference card</b>                              | <b>14</b> |

---

## 1 Overview

`picoacq` is the data-acquisition companion to `triloop`, the analysis package for three-loop magnetic-field-array HF polarimetry. `picoacq` drives a Pico Technology PicoScope 5444D (or other 5000-series digitiser) over USB, captures four channels of coherently sampled IQ-style real signals, and writes them to an HDF5 file format that `triloop` reads.

This manual covers acquisition only. For analysis, polarimetry, and the underlying magnetoionic physics, see the companion documents *triloop User Manual* and *Three-Loop Magnetic-Field Array Technical Report*.

### 1.1 What `picoacq` does

1. Detects an attached PicoScope 5000-series digitiser via Pico’s USB driver, opens it at the appropriate vertical resolution (Section 4).
2. Configures the user-selected channels for input range, coupling, and sample rate (Sections 5–6).
3. Triggers a block-mode acquisition of the requested duration, reads the raw 16-bit ADC samples back to host memory, converts them to volts, and writes the result to an HDF5 file with complete metadata (Section 8).
4. In continuous-monitor mode, repeats this loop on a fixed cadence to build a long-running dataset suitable for ionospheric studies.

If no PicoScope is attached or the SDK is missing, `picoacq` falls back to a synthetic data simulator so the analysis chain can be tested without hardware.

### 1.2 Audience

This manual assumes basic Python familiarity, comfort with command-line tools, and a working knowledge of HF radio signal capture. The reader should be familiar with concepts like sample rate, voltage range, ADC resolution, and the difference between block and streaming acquisition.

## 2 Hardware setup

### 2.1 The PicoScope 5444D

The 5444D is a 4-channel 200 MHz USB-connected oscilloscope with a variable-resolution (“FlexRes”) ADC. Specifications relevant to HF capture:

- **4 analog input channels**, BNC, switchable between  $1\text{ M}\Omega \parallel 15\text{ pF}$  and  $50\text{ }\Omega$  termination.
- **Variable resolution**, 8 to 16 bits. See Section 4 for the trade-off between resolution and channel count.
- **Synchronous sampling** of all enabled channels. All four channels share a single sample clock and trigger. Channel-to-channel sample skew is below one sample period.
- **Voltage ranges** from  $\pm 10\text{ mV}$  to  $\pm 20\text{ V}$  in 11 steps.
- **Streaming throughput** over USB 3.0:  $\sim 400\text{ MB/s}$  sustained.
- **Onboard memory**: 512 MS total, split across enabled channels. In 4-channel mode each channel gets 128 MS; in  $\leq 2$ -channel mode each gets 256 MS. This sets the maximum block-mode capture length (8.2 s for 4 channels at the `picoacq` default rate of  $15.625\text{ MS/s}$ ).
- **No external reference clock input**. The 5444D enclosure has only the four BNC signal inputs; there is no separate 10 MHz reference jack. A GPS-disciplined oscillator can still be used as a reference by injecting its 10 MHz pilot tone into one (or all) of the input channels through a signal combiner — the recovered pilot then disciplines timing in software.

### 2.2 The crucial connection check

Before installing any software, plug the PicoScope into the host via its USB-3 cable and visually verify the LED indicator lights up green (power) and the host OS sees the device as a USB peripheral. On macOS:

```
system_profiler SPUSBDataType | grep -i pico
```

Should print at least one match line. If nothing prints, the device is not detected — check the USB cable, port (must be USB 3.0+, not 2.0), and that the LED is illuminated.

### 2.3 Sanity-test with the GUI app

Before driving the scope from `picoacq`, run Pico’s free GUI application `PicoScope.app` (download from <https://www.picotech.com/downloads>). This rules out any hardware issue independent of `picoacq` code.

The GUI is also useful for:

- **Choosing voltage ranges** per channel by watching live time-domain traces from your loop preamps and whip; pick a range that uses most of the dynamic range without clipping.
- **Checking signal presence** via the live FFT spectrum view, e.g. confirming that WWV at the expected  $\sim 2\text{ kHz}$  IF offset is visible above the noise floor.
- **Trigger debugging** interactively before scripting.

**Important:** the GUI app and `picoacq` cannot both hold the device handle simultaneously. **Quit `PicoScope.app` before invoking `picoacq capture`** or the SDK call will fail with a “device in use” error.

## 3 Software installation

picoacq requires three software pieces. All three are independent and must be installed separately on macOS, Linux, or Windows.

### 3.1 Layer 1: PicoSDK system library

The PicoSDK is a system-level shared library installed via Pico's own installer. It provides the C-level functions `ps5000aOpenUnit`, `ps5000aSetChannel`, etc. that all higher-level wrappers eventually call.

1. Visit <https://www.picotech.com/downloads>.
2. Select **PicoSDK** (NOT **PicoScope**, which is the GUI app).
3. Choose your operating system and architecture.
4. Install the resulting `.pkg` (macOS), `.deb` / `.rpm` (Linux), or `.exe` (Windows).

**Where the library lands on macOS:** as of 2026, Pico packages the dylibs as a framework at `/Library/Frameworks/PicoSDK.framework/Libraries/`. Each driver lives in its own subdirectory (e.g. `libps5000a/libps5000a.dylib`). This is the modern macOS install layout, distinct from the older flat `/usr/local/lib/` location.

To verify successful install:

```
sudo find / -name "libps5000a*" 2>/dev/null
```

This should list at least one `libps5000a.dylib` (or platform equivalent).

### 3.2 Layer 2: picosdk Python wrapper

A thin ctypes wrapper around the system library, distributed via PyPI:

```
python3 -m pip install picosdk
```

**Common pitfall on macOS:** the system has multiple Python installations (a 3.9 from Apple's macOS, and a 3.13 from [python.org](https://python.org)). `pip` and `pip3` may resolve to different installations. Use the `-m pip` form to force the binding:

```
python3 -m pip install picosdk
```

Verify:

```
python3 -c "from picosdk.ps5000a import ps5000a; print('SDK OK')"
```

### 3.3 Layer 3: picoacq itself

Editable install from the project tree:

```
cd <project-root>
pip install -e ./picoacq
```

This installs the `picoacq` command-line tool and Python package.

### 3.4 Verifying the install

```
picoacq --help
picoacq capture --simulate -o /tmp/test.h5 --duration 0.1
```

The first should print a help message; the second should write a synthetic capture file using the simulator. No real hardware needed for either.

`picoacq` automatically detects the macOS Framework path and adds it to `DYLD_LIBRARY_PATH` at import time, so the common “library not found” error on macOS is handled transparently. On Linux you may need to add the SDK’s library directory to `LD_LIBRARY_PATH` manually if the installer didn’t.

## 4 Hardware capabilities and resolution selection

The 5444D’s FlexRes ADC trades vertical resolution against the number of simultaneously-enabled channels and the per-channel sample rate. Knowing which mode you’re in is essential to avoid the “`PICO_INVALID_NUMBER_CHANNELS_FOR_RESOLUTION`” error when configuring your acquisition.

| Resolution | Max channels | Max rate per ch                 | Use for                       |
|------------|--------------|---------------------------------|-------------------------------|
| 8-bit      | 4            | 1 GS/s (1 ch) / 250 MS/s (4 ch) | High bandwidth, low precision |
| 12-bit     | 4            | 500 MS/s                        | General oscilloscope use      |
| 14-bit     | 4            | 125 MS/s                        | <b>HF triloop polarimetry</b> |
| 15-bit     | 2            | 125 MS/s                        | 2-ch high-precision           |
| 16-bit     | 2            | 62.5 MS/s                       | Ultra-precision, 2-ch only    |

Table 1: PicoScope 5444D resolution / channel-count compatibility. Note: 16-bit mode is restricted to 2 channels. For 4-channel triloop applications, 14-bit is the highest available resolution.

**For triloop polarimetry with 3 loops + 1 whip = 4 channels, `picoacq` automatically selects 14-bit resolution.** This gives you 86 dB of dynamic range, which is far more than necessary for HF work after narrow-band filtering. The 16-bit mode that some Pico documentation references is a 2-channel-only mode and is not used by `picoacq`.

If you have only 2 (or fewer) channels enabled, `picoacq` auto-selects 16-bit resolution. This selection happens in `picoacq.ps5444a.open_5444()` based on the channel count passed by the recorder.

## 5 Channel configuration

Each enabled channel needs three settings: a **voltage range** (per-channel,  $\pm 10$  mV to  $\pm 20$  V), a **coupling mode** (AC or DC), and an **enable bit** (`picoacq` sets this automatically for any channel listed in `--channels`).

### 5.1 Voltage range selection

The voltage range determines the ADC’s full-scale span. The 14-bit ADC has  $2^{14} = 16384$  codes; for a  $\pm 2$  V range the code-to-volts factor is  $4/16384 \approx 244 \mu\text{V}/\text{code}$ .

**Rule of thumb:** pick a range so the largest expected signal peak uses about 50–80% of full scale. Too small a range clips the signal and produces nonlinearities; too large a range wastes ADC bits on noise.

For HF triloop work with active mag-loops and a vertical mini-whip, typical signal levels at the receiver after amplification are 50 mV to 1 V peak.  $\pm 2$  V is usually a safe default.

## 5.2 Auto-ranging

By default `picoacq` performs an **auto-range probe** before each capture, eliminating the need to know your signal levels in advance. The procedure:

1. Open the scope at the maximum range ( $\pm 20$  V) on every channel.
2. Take a brief probe capture (default 100 ms at the requested sample rate).
3. For each channel: measure the peak  $|x|$  and the RMS of the captured samples in volts.
4. For each channel: pick the smallest available Pico voltage range whose full scale  $\geq K \cdot \text{peak}$  (default  $K = 3$ , i.e. 10 dB headroom on top of the measured peak).
5. Re-configure each channel at the chosen range and run the science capture.

The Pico’s available ranges are quantised:  $\pm 10$  mV,  $\pm 20$ ,  $\pm 50$ ,  $\pm 100$ ,  $\pm 200$ ,  $\pm 500$  mV,  $\pm 1$ ,  $\pm 2$ ,  $\pm 5$ ,  $\pm 10$ ,  $\pm 20$  V. The auto-range chooser always rounds *up* to the next available range, so the actual headroom may exceed  $K$  by up to a factor of  $\sim 2$  depending on where your peak falls between range steps.

**What auto-ranging buys you.** At the maximum range ( $\pm 20$  V), a 50 mV peak signal occupies only 0.25% of full scale. With a 14-bit ADC this means only  $\sim 5$  effective bits of useful resolution =  $\sim 32$  dB quantization-limited SNR. Auto-ranging this same signal to (say)  $\pm 200$  mV yields a  $5\times$  headroom and gives you the full 14 bits across the active range:  $\sim 78$  dB SNR. That’s a  $\sim 46$  dB improvement in dynamic range from one decision.

**Disabling auto-range.** If you want to set ranges manually (e.g., when calibrating against a known signal source where you control the input amplitude), pass `--no-auto-range` to the CLI:

```
# Manual ranges (--range applies to all enabled channels):
picoacq capture -o cap.h5 --duration 8 \
    --channels A,B,C,D --range 0.5 --no-auto-range
```

**Tuning the headroom factor.** The default  $K = 3$  is moderately conservative (10 dB above peak). For very stable signals where transient excursions are unlikely, you can try  $K = 2$  to gain  $\sim 3$  dB more dynamic range. For unpredictable signals (e.g., picking up sferics from nearby thunderstorms),  $K = 5$  to 10 is safer. Override with `--headroom`:

```
picoacq capture --headroom 5 ...
```

**Probe-result metadata.** The HDF5 file’s `capture_settings` dict includes:

- `auto_range_enabled`: True / False
- `headroom`: the  $K$  factor used
- `auto_range_probe`: the per-channel `peaks_volts` and `rms_volts` measured during the probe
- `picoscope_ranges_volts`: what the Pico’s range was actually set to (after rounding up to the available range)

This lets you reconstruct the dynamic range used after the fact.

### 5.3 Coupling

DC coupling preserves the full signal including any DC offset. AC coupling adds a series capacitor that high-passes at  $\sim 1$  Hz.

For HF capture, DC and AC are both fine. AC removes any DC offset from the active electronics, which is useful for keeping the ADC centered. Default in `picoacq` is DC.

### 5.4 Channel-name to PicoScope-port mapping

The PicoScope 5444D has BNC connectors labelled A, B, C, D. By convention in `triloop`:

- A: loop L1 (north-of-up)
- B: loop L2 ( $+120^\circ$ )
- C: loop L3 ( $-120^\circ$ )
- D: vertical whip (or other reference)

You may rearrange via `loops_config` in your analysis script.

## 6 Sample-rate selection

The PicoScope 5000-series uses an integer “timebase” index to specify sample period. At 14-bit resolution with 4 channels active, the available sample periods are quantized to specific values; you *request* a target rate, and the SDK returns the closest available.

### 6.1 Default rate: 15.625 MS/s, all WWV bands at once

`picoacq`’s default sample rate is **15.625 MS/s** in 4-channel mode. This rate was chosen because it is high enough that *all five WWV bands* (2.5, 5, 10, 15, and 20 MHz) fold to non-overlapping baseband locations through bandpass undersampling, so a single capture digitises every band simultaneously and coherently across all four input channels.

Under bandpass undersampling, an RF tone at  $f_{\text{rf}}$  aliases to baseband as

$$f_{\text{bb}} = |f_{\text{rf}} - n \cdot f_s|, \quad n = \text{round}(f_{\text{rf}}/f_s).$$

The Nyquist zone  $z = \lfloor 2f_{\text{rf}}/f_s \rfloor + 1$  determines whether the spectrum is preserved (odd zones) or frequency-inverted (even zones). At  $f_s = 15.625$  MS/s the WWV bands map as follows:

The helper `picoacq.alias.alias_of(f_rf, f_s)` computes this mapping programmatically; `rf_for_baseband()` performs the inverse lookup given a list of expected RF bands (typically the `rf_bands_hz` field stored in the file’s `capture_settings`).

| RF (MHz) | Nyquist zone | Baseband (MHz) | Inverted? |
|----------|--------------|----------------|-----------|
| 2.5      | 1            | 2.500          | no        |
| 5.0      | 1            | 5.000          | no        |
| 10.0     | 2            | 5.625          | yes       |
| 15.0     | 2            | 0.625          | yes       |
| 20.0     | 3            | 4.375          | no        |

Table 2: WWV bands at  $f_s = 15.625$  MS/s. All five bands occupy distinct, non-overlapping baseband locations. Bands in even Nyquist zones come out spectrum-inverted, which the analysis tools handle via complex conjugation of the appropriate down-converted slice.

**Why not 12.5 MS/s?** An apparently-tempting lower rate of 12.5 MS/s has the WWV 5 and 10 MHz bands both folding to 2.5 MHz baseband on top of the unaliased 2.5 MHz signal — three bands pile up in the same baseband bin and become inseparable. 15.625 MS/s is the lowest rate at which the five WWV bands separate cleanly.

**Why not 62.5 MS/s direct?** Sampling above  $2 \cdot 20$  MHz puts every WWV band in zone 1 (no aliasing), but the 128 MS/channel buffer then holds only  $\sim 2$  s of data. 15.625 MS/s preserves absolute phase information at all five bands while extending the maximum block to 8.2 s — the longer block dominates phase-stability science requirements ( $\tau \leq 1$  s coherence times).

## 6.2 The `--rf-bands` flag

By default, `picoacq capture` and `picoacq monitor` record the WWV band list `2.5e6,5e6,10e6,15e6,20e6` into the file’s `capture_settings` as `rf_bands_hz`. Override with `--rf-bands` for a different observation plan (e.g. CHU at 3.33, 7.85, 14.67 MHz):

```
picoacq capture --rf-bands "3.33e6,7.85e6,14.67e6" ...
```

Pass an empty string to omit the metadata entirely. Downstream tools read this field to map detected baseband peaks back to the originating RF band.

## 6.3 Common sample rates and their timebase indices

For 4-channel 14-bit operation:

| Timebase | Sample period | Sample rate | Useful for                                     |
|----------|---------------|-------------|------------------------------------------------|
| 3        | 64 ns         | 15.625 MS/s | <b>picoacq default</b> ; all WWV bands at once |
| 4        | 96 ns         | 10.42 MS/s  |                                                |
| 8        | 128 ns        | 7.81 MS/s   |                                                |
| 512      | 4.08 $\mu$ s  | 245 kHz     | Single-band IF capture                         |
| 1024     | 8.18 $\mu$ s  | 122 kHz     | Long captures, less data                       |
| 4000     | 31.98 $\mu$ s | 31.3 kHz    | Audio-rate analysis                            |

Table 3: Common timebase indices for 4-channel 14-bit acquisition on the 5444D. Timebase indices below 3 are invalid in 4-channel mode and are skipped. See the SDK reference for the full table.



`picoacq capture -rate jHz` accepts any positive value and selects the timebase whose actual rate is at or below your request. The rate *actually used* is recorded in the HDF5 metadata as `sample_rate`. Don't assume your requested rate equals the actual rate; always check the file's metadata.

## 6.4 Memory budget and block duration limits

Samples are stored as float32 (4 bytes each) in volts. A **block-mode capture cannot exceed the per-channel onboard memory**; `picoacq` validates the requested duration against the buffer up front and refuses with a clear error if you ask for too much:

| Channels | Buffer per ch | Max block @ 15.625 MS/s |
|----------|---------------|-------------------------|
| 4        | 128 MS        | 8.19 s                  |
| 2        | 256 MS        | 16.38 s                 |
| 1        | 512 MS        | 32.77 s                 |

Table 4: Onboard sample budget per channel and the resulting maximum block duration at the default rate.

A typical 8 s capture at 15.625 MS/s with 4 channels stores:

$$8 \cdot 15,625,000 \cdot 4 \cdot 4 = 2,000 \text{ MB}$$

on disk before compression. HDF5's gzip cuts this  $\sim 30\%$  for typical signals, but at this volume host RAM is the limiting factor. For long captures or higher rates, switch to streaming mode (not yet implemented in `picoacq`; planned for v0.2).

## 7 Command-line interface

`picoacq` ships a single CLI tool with two main subcommands: `capture` and `monitor`. Both produce HDF5 capture files in the format described in Section 8.

### 7.1 One-shot capture: `picoacq capture`

```
picoacq capture -o cap.h5 \
  --duration 8 \           # seconds (max 8.19 at 4ch/15.625 MS/s)
  --rate 15625000 \        # samples per second (default)
  --channels A,B,C,D \     # comma-separated, no spaces
  --range 2.0 \           # +/- volts (used only if --no-auto-range)
  --coupling DC            # or AC
```

All flags above are at their defaults except `-o`; bare `picoacq capture` produces an 8-second 4-channel 15.625 MS/s capture written to the cwd.

**Output path defaults.** If `--output` (or `-o`) is omitted, the file is written into the **current working directory** as `capture_<UTC-timestamp>.h5`. This makes `cd` into a directory on an external disk and running `picoacq capture` the natural way to deposit data there:

```
cd /Volumes/MyExternalDisk/triloop_data/
picoacq capture --duration 8 --rate 15625000 \
  --channels A,B,C,D
# -> writes /Volumes/MyExternalDisk/triloop_data/capture_20260523T173951Z.h5
```

If you pass an explicit `-o cap.h5`, that's interpreted relative to your current directory. Pass an absolute path (`-o /Volumes/Disk/cap.h5`) to override the cwd-relative default.

If the PicoSDK or hardware is unavailable, `picoacq` prints a warning and falls back to its simulator (Section 7.3). A successful real-hardware capture writes only the file path to stdout and creates the HDF5 file.

## 7.2 Continuous monitoring: `picoacq monitor`

For long unattended runs, `picoacq monitor` takes one capture every `--interval` seconds and writes a timestamped file:

```
picoacq monitor -d /data/triloop_runs/ \
  --interval 600 \           # seconds between capture starts
  --duration 8 \            # length of each capture
  --rate 15625000 \         # all WWV bands at once
  --prefix wwv_             # filename prefix
```

Output filenames look like `wwv_20260520T144312Z.h5`.

**Output directory defaults.** Like `capture`, the `--out-dir` option for `monitor` defaults to the current working directory (`.`). This means a typical workflow is:

```
cd /Volumes/ExternalDisk/wwv_run_2026_05/
picoacq monitor --interval 600 --duration 8 --rate 15625000 \
  --channels A,B,C,D --prefix wwv_
# -> writes wwv_<timestamp>.h5 files into the external disk
```

Pass `-d /some/other/path` to override. `SIGTERM` and `SIGINT` are handled cleanly — the current capture finishes and the loop exits. Pass `--n-captures N` to stop after  $N$  captures rather than running indefinitely.

## 7.3 Simulator mode: `picoacq capture --simulate`

The simulator generates a synthetic three-loop dataset with controllable arrival direction, polarization, additive Gaussian noise, and Faraday rotation rate. It writes the same HDF5 format as the real-hardware path so `triloop` can analyze the result without modification.

```
picoacq capture -o cap_sim.h5 --simulate \
  --sim-pol rcp \           # linear_vertical, linear_horizontal,
                             # rcp, lcp, elliptical
  --sim-faraday 30 \        # deg/s Faraday rotation rate
  --sim-snr 25 \            # per-channel SNR (dB)
  --sim-az 273 \           # arrival azimuth (deg from N CW)
  --sim-el 12 \            # arrival elevation (deg)
  --sim-carrier 25000       # IF carrier freq (Hz)
```

The simulator is also used as the no-hardware fallback in real-capture mode. This is useful for code development on a laptop without a scope attached.

## 8 HDF5 file format

picoacq writes a self-describing HDF5 file (format version 0.1) that is also the input format for triloop.

```
/                                HDF5 root
  @format_version                "0.1"
/metadata                       group, with the following attributes:
  @sample_rate                   float64, Hz (actual achieved rate)
  @start_time_utc                ISO-8601 string, e.g. "2026-05-23T17:33:37.994Z"
  @duration_s                    float64
  @scope_model                   string, e.g. "PicoScope 5444D"
  @scope_serial                  string
  @capture_settings              JSON-encoded dict (full kwargs)
  @loops_config                  JSON-encoded dict (loop geometry)
/channels                       group containing per-channel datasets
  /A                             float32 array, length N (in volts)
  /B                             (same)
  /C                             (same)
  /D                             (optional reference channel)
  Each channel dataset has attributes:
    @gain_v_per_count            float64 (1.0; values are already in volts)
    @offset_v                    float64 (typically 0.0)
/time                           optional float64 dataset, length N
                                (regenerated from sample_rate if absent)
```

The format is intentionally simple — a third party can write or read it with  $\sim 30$  lines of Python using h5py.

### 8.1 Reading a capture file in Python

```
import h5py, json, numpy as np

with h5py.File("cap.h5", "r") as f:
    sr = float(f["metadata"].attrs["sample_rate"])
    duration = float(f["metadata"].attrs["duration_s"])
    channels = {name: f["channels"][name][:].astype(np.float64)
                 for name in f["channels"].keys()}
    loops_config = json.loads(f["metadata"].attrs["loops_config"])

t = np.arange(len(next(iter(channels.values())))) / sr
print(f"Loaded {len(channels)} channels, {len(t)} samples each, "
      f"at {sr:.1f} Hz over {duration:.2f} s")
```

For more complex use, `triloop.io.hdf5.read_capture()` provides a tested implementation that returns a dict with all metadata already parsed.

## 9 Worked examples

### 9.1 Example 1: smoke-test on the bench

After installation, with the PicoScope plugged in via USB and PicoScope.app not running:

```
# 0.5-second snapshot, all 4 channels, auto-range (default)
picoacq capture --duration 0.5 --channels A,B,C,D
```

```
# Inspect the result
triloop summary capture_*.h5
```

The auto-range probe runs first, then the science capture:

```
[picoacq] auto-range probe (100 ms at 15625000 Hz, headroom=3.0)
[picoacq] probe results:
      A: peak=230.11 mV  RMS=107.27 mV  -> range +/- 1.0 V
      B: peak=234.99 mV  RMS=113.19 mV  -> range +/- 1.0 V
      C: peak=272.23 mV  RMS=126.72 mV  -> range +/- 1.0 V
      D: peak=366.83 mV  RMS=183.37 mV  -> range +/- 2.0 V
wrote /Users/.../capture_20260523T173951Z.h5
```

triloop summary confirms:

```
file:          capture_20260523T173951Z.h5
format_version: 0.1
start_time_utc: 2026-05-23T17:33:37.994093+00:00
sample_rate:   209732 Hz
duration_s:    0.4768 s
scope:         PicoScope 5444D (s/n EW123/4567)
channels:
  A: n=100000, range=[-0.005, +0.023]
  B: n=100000, range=[-0.003, +0.024]
  C: n=100000, range=[-0.003, +0.027]
  D: n=100000, range=[-0.001, +0.018]
```

The voltage ranges shown by `summary` are the actual **captured-signal** range during the science capture, which is typically much smaller than the auto-ranged Pico setting (because the auto-range probe captured a different instant of input transients, and we still rounded up to a quantized Pico range).

The actual achieved sample rate may differ slightly from the requested rate — the SDK selects the nearest valid timebase index at or below the request (Section 6). Always consult `metadata/sample_rate` in the file rather than the requested value when computing time axes.

## 9.2 Example 2: 8-second all-WWV-band capture

Connect a triloop array (3 mag-loops to channels A-C) plus a vertical whip (channel D), with each input from an active preamp. Take an 8-second capture at the default 15.625 MS/s, which simultaneously digitises all five WWV bands (Section ??):

```
picoacq capture -o /data/triloop/cap_wwv_$(date -u +%Y%m%dT%H%M%SZ).h5 \
  --duration 8 --channels A,B,C,D
```

Resulting file size: about 1.4 GB compressed (raw is 2.0 GB). With  $\sim 125$  M samples per channel over 8 seconds, the WWV bands appear at the baseband locations listed in Table ??.

Run triloop analysis on it:

```
triloop analyze cap_wwv_20260523T173337Z.h5 \
  --rf-band 5e6 \           # one of the recorded rf_bands_hz entries
  --bw 4000 \
  --az 9 --el 35           # FC->APD geometry
```

triloop reads the file's `rf_bands_hz` setting, maps the requested RF band through `picoacq.alias` to its baseband position, and slices that band out for analysis.

### 9.3 Example 3: continuous overnight monitor at APO

Run continuously, 8 seconds out of every 10 minutes:

```
picoacq monitor -d /mnt/data/wwv_monitor/ \
  --interval 600 --duration 8 --channels A,B,C,D \
  --prefix wwv_
```

Files appear as `wwv_20260523T173337Z.h5`, `wwv_20260523T174337Z.h5`, etc. At ~1.4 GB per file this is about 8.5 GB/hour, or ~200 GB per 24-hour observation — pre-allocate disk accordingly.

The monitor catches SIGINT and SIGTERM cleanly, finishing the current capture before exiting. Use `Ctrl-C` or `kill -TERM PID` to stop.

## 10 Troubleshooting

### 10.1 Common errors and what they mean

[picoacq] real capture failed: picosdk not installed The picosdk Python wrapper is missing. Run `python3 -m pip install picosdk`. Note the use of `python3 -m pip` rather than `pip` — on macOS the two may resolve to different Python installations.

[picoacq] real capture failed: PicoSDK (ps5000a) not found, check `LD_LIBRARY_PATH` The system-level PicoSDK library isn't installed, or picoacq's framework-path detection didn't find it. Run `sudo find / -name "libps5000a*" 2>/dev/null` to confirm the library is present on the system. If it's there but picoacq isn't finding it, manually set the path:

```
export DYLD_LIBRARY_PATH=/Library/Frameworks/PicoSDK.framework/Libraries/
libps5000a:$DYLD_LIBRARY_PATH
```

**PICO\_NOT\_FOUND** The PicoScope hardware isn't visible to the SDK. Possible causes:

- PicoScope.app or another program is holding the device handle. Quit those.
- USB cable is bad or unplugged.
- Pico's user-space USB plugin wasn't authorised on macOS. Re-run the installer or check System Preferences → Security and authorise the Pico kext/extension.

**PICO\_INVALID\_NUMBER\_CHANNELS\_FOR\_RESOLUTION** You're trying to use 4 channels at 16-bit resolution. The 5444D supports at most 2 channels at 16-bit; for 4 channels, you need 14-bit. `picoacq` normally auto-selects this; if you've manually overridden the resolution, choose 14-bit.

`argument N: TypeError: '_ctypes.CArgObject' object cannot be interpreted as an integer`  
A ctypes argument-type mismatch in the picosdk Python wrapper. Should not occur with current `picoacq`; if you see it, it likely means the picosdk version on your system has function signatures that differ from what `picoacq` expects. File an issue with the SDK version (`python3 -c "import picosdk; print(picosdk.__version__)"`).

could not find a timebase for X Hz The requested sample rate isn't achievable with the current channel count and resolution. At 4-channel 14-bit, valid sample rates start around 31 Hz (timebase 4000) and go up to 125 MS/s (timebase 3). If you really need a sample rate `picoacq` reports as unreachable, try reducing the channel count or check the PicoScope 5000-series Programmer's Guide for valid timebase indices.

## 10.2 Verifying timebase availability manually

To diagnose timebase issues, this snippet walks the available indices for your current channel count:

```
import ctypes
from picoacq.ps5444a import open_5444, configure_channel
from picosdk.constants import PICO_STATUS_LOOKUP

with open_5444(n_channels=4) as (ps, handle):
    for ch in ['A', 'B', 'C', 'D']:
        configure_channel(ps, handle, ch, range_volts=2.0)
    print('all 4 channels configured')
    for tb in [3, 4, 8, 16, 64, 256, 512, 1024, 4000]:
        timeIntervalNs = ctypes.c_float()
        maxSamples = ctypes.c_int32()
        ret = ps.ps5000aGetTimebase2(handle,
                                     ctypes.c_uint32(tb), ctypes.c_int32(1024),
                                     ctypes.byref(timeIntervalNs),
                                     ctypes.byref(maxSamples),
                                     ctypes.c_uint32(0))
        if ret == 0:
            rate = 1.0 / (timeIntervalNs.value * 1e-9)
            print(f"  tb={tb}: dt={timeIntervalNs.value:.3f} ns, "
                  f"rate={rate:.0f} Hz")
        else:
            print(f"  tb={tb}: status={ret} ({PICO_STATUS_LOOKUP[ret]}")
```

## 11 Hardware capture process, step by step

For reference, here's the sequence of SDK calls `picoacq` makes to perform one capture. This is the part of `picoacq.ps5444a` that ultimately runs:

1. `ps5000aOpenUnit`: opens the device handle. Resolution argument is set based on channel count (14-bit for 4-channel mode, 16-bit for  $\leq 2$ ).
2. For each enabled channel: `ps5000aSetChannel` configures the channel for the requested voltage range and coupling (DC/AC).
3. `ps5000aGetTimebase2`: walks timebase indices and finds the one whose sample rate is at or just below the requested target rate. Some early indices are invalid in 4-channel mode and are skipped.
4. For each enabled channel: `ps5000aSetDataBuffer` allocates a host-side buffer and tells the driver where to write samples.
5. `ps5000aRunBlock`: triggers the block acquisition. The driver fills the buffers and signals completion.

6. Poll `ps5000aIsReady` every 5 ms until the driver reports the block is complete.
7. `ps5000aGetValues`: pulls the captured samples into the pre-allocated host buffers.
8. Convert int16 ADC codes to volts using the per-channel range setting ( $\text{gain} = \text{range\_volts} / 32767$ ).
9. `ps5000aCloseUnit`: releases the device handle.

The total time for a 1-second capture at 200kSPS is about 1.5 seconds (including 0.5 s of overhead for opening, configuring, and closing). For longer captures the overhead is small.

## 12 Limitations of v0.1

- **Block mode only.** Continuous streaming over USB at hundreds of MB/s is not yet implemented. For now, captures are finite-duration block mode reads. This is fine for typical monitor cadences (1 minute / 10 minutes) but not for uninterrupted multi-hour streaming.
- **Same manual-range value across all channels** (when `--no-auto-range` is used). The CLI takes a single `--range` value applied to every channel. Auto-range always picks per-channel ranges independently. Per-channel manual ranges are supported in the underlying `configure_channel()` function, just not exposed in the CLI.
- **No external trigger configuration.** The current code uses a software trigger immediately on `RunBlock`. External hardware-trigger support is on the roadmap for v0.2.
- **Serial number reads as a placeholder.** The metadata records “(read from scope handle)” rather than the actual unit serial. We have the SDK call needed (`ps5000aGetUnitInfo`); just not wired up yet.
- **No software pilot-tone discipline yet.** The 5444D has no external 10 MHz reference jack; the planned workflow is to inject a GPSDO 10 MHz pilot into one of the input channels and correct the recorded signals’ time base in software at analysis time. That correction is not yet implemented in `picoacq` or `triloop`.

These will be addressed in subsequent releases. For most `triloop` science applications, the v0.1 feature set is sufficient.

## 13 Quick reference card

```
# Install (one time)
python3 -m pip install picosdk
pip install -e <project>/picoacq

# Quick smoke test
picoacq capture --simulate -o /tmp/test.h5 --duration 0.5

# Single capture (auto-range by default; all WWV bands; written to cwd)
picoacq capture                                # 8 s, 4 ch, 15.625 MS/s

# Single capture, manual range, explicit output
picoacq capture -o cap.h5 --duration 8 \
    --channels A,B,C,D --no-auto-range --range 2.0
```

```
# Continuous monitor (auto-range each capture, written to cwd)
picoacq monitor --interval 600 --duration 8 --channels A,B,C,D

# CHU bands instead of WWV (override the default --rf-bands list)
picoacq capture --rf-bands "3.33e6,7.85e6,14.67e6"

# Stop monitor cleanly: Ctrl-C
```

```
# Read a file in Python
import h5py
with h5py.File("cap.h5", "r") as f:
    sr = float(f["metadata"].attrs["sample_rate"])
    A = f["channels"]["A"][:]
```